

Visual Debugger: Synaptic Overlap

Project Documentation

March 29, 2026

1 Description

The `debug_plot_synaptic_overlap` function serves as a critical visual sanity check for the connectivity model. When performing complex 3D bounding-box mathematics and shifting arrays by physical coordinates, it is remarkably easy to invert a sign or axis, causing neurons to connect mathematically while missing each other in physical space.

Instead of rendering a computationally heavy 3D volume, this function utilizes a classic microscopy technique to flatten 3D data into crisp 2D images. This allows users to instantly verify that the pre-synaptic axon and post-synaptic dendrite intersect exactly where the algorithm predicts.

2 Technical Breakdown

2.1 1. Loading the Raw Volumes

The process begins by loading the raw `.npy` arrays for the pre-synaptic axon and post-synaptic dendrite. At this stage, the data exists as full 3D grids of spatial probabilities.

2.2 2. Squashing 3D to 2D (Maximum Intensity Projection)

To simplify the visualization, the function employs `np.max(grid, axis=...)` to create a **Maximum Intensity Projection (MIP)**:

- **XY Plane (Top-Down):** The engine looks down the *Z*-axis (`axis=2`) and identifies the highest probability value in each vertical column, squashing it into a 2D pixel.
- **XZ Plane (Side View):** The engine looks across the *Y*-axis (`axis=1`) and flattens the volume against the “wall.”

This provides a structural footprint of the spanning trees without the visual noise of a 3D cloud.

2.3 3. Masking for Transparency

To prevent empty space (zeros) in the top layer from obscuring the layer beneath, the function uses:

```
np.ma.masked_where(grid <= 0, grid)
```

This converts all zero-value voxels into transparent pixels, allowing the blue dendrite and orange axon to visually blend like colored glass.

2.4 4. Aligning Math to the Physical World

The function must map array indices to physical space using the `get_extent` helper. This calculates boundaries ($[x_{min}, x_{max}, y_{min}, y_{max}]$) by taking the soma position (the center of the array) and stretching the image outward by exactly half the grid size multiplied by the `voxel_size`. This ensures an array index of $[0, 0]$ correctly maps to its intended physical coordinate (e.g., $-500\ \mu\text{m}$).

2.5 5. Rendering the Multi-Layer Canvas

The final output is a two-panel figure (Top-Down and Side View) composed of several layers:

1. **The Spanning Trees:** `ax.imshow` renders the blue dendrite and orange axon extents.
2. **The Soma Markers:** Precise x, y, z coordinates of the somas are plotted as a blue triangle and a red circle to confirm array centering.
3. **The Synaptic Heatmap:** The `overlap_centroids` (mathematical intersections) are plotted as square markers. By passing `overlap_probabilities` into the color argument (`c=`), Matplotlib generates a **viridis** gradient heatmap, highlighting voxels with the highest joint probability.

3 Interpretation

A successful simulation is indicated by a dense cluster of bright yellow/green squares situated perfectly within the physical intersection of the orange (axonal) and blue (dendritic) clouds.

4 Usage Example

```
import matplotlib.pyplot as plt
from engine import debug_plot_synaptic_overlap

# Generate the sanity check plot
fig = debug_plot_synaptic_overlap(
    pre_id, post_id,
    centroids, probabilities,
    voxel_size=10.0
)
plt.show()
```